```
# -*- coding: utf-8 -*-
"""
MODELO TRIBOLÓGICO INTEGRADO - V7.81
Pydroid 3 / Android / Tkinter

V0.2:
- superfícies paramétricas gerais r1(u,v,theta),
r2(xi,eta,theta)
- derivadas numéricas das superfícies
- normais
- velocidade relativa por evolução em theta
- teste de coincidência geométrica
- teste cinemático v_rel . n = 0
- identificação numérica de candidatos à linha de contato
- construção da representação do volume tribológico em
torno dos candidatos

IMPORTANTE:
Esta versão NÃO inventa uma geometria física específica.
A geometria deve ser fornecida pelo usuário como seis
expressões:
x1(u,v,theta), y1(...), z1(...)
x2(xi,eta,theta), y2(...), z2(...)

Convenção:
- u, v, xi, eta: parâmetros geométricos
- theta: posição cinemática
- theta_dot: velocidade angular de theta [rad/s]
- coordenadas internas: metros
"""
```

```
import math
import tkinter as tk
from tkinter import ttk, messagebox


#
============================================================
===
# V7.77 — INTERFACE MOBILE CONSOLIDADA
#
============================================================
===

V774_NAVIGATION = [
    ("ENTRADA", "page_input"),
]

def v774_mobile_navigation():
    """Navegação consolidada para telas pequenas; não
altera o núcleo matemático."""
    return list(V774_NAVIGATION)


#
============================================================
===
# ERROS
#
============================================================
===

class MissingInputError(Exception):
    pass
```

```
class ValidationError(Exception):
    pass


#
=============================================================
===
# AMBIENTE MATEMÁTICO CONTROLADO
#
=============================================================
===

SAFE_MATH = {
    "sin": math.sin,
    "cos": math.cos,
    "tan": math.tan,
    "asin": math.asin,
    "acos": math.acos,
    "atan": math.atan,
    "atan2": math.atan2,
    "sqrt": math.sqrt,
    "exp": math.exp,
    "log": math.log,
    "pi": math.pi,
    "abs": abs,
    "pow": pow,
}


#
=============================================================
```

```
===
# VETORES
#
============================================================
===


#
============================================================
===
# SUPERFÍCIE PARAMÉTRICA
#
============================================================
===


#
============================================================
===
# PONTO AMOSTRADO
#
============================================================
===


#
============================================================
===
# CONTATO CINEMÁTICO
#
============================================================
===
```

```
#
============================================================
===
# AMOSTRAGEM
#
============================================================
===


#
============================================================
===
# BUSCA DE CANDIDATOS
#
============================================================
===


#
============================================================
===
# VOLUME TRIBOLÓGICO
#
============================================================
===


#
============================================================
===
# INTERFACE
#
```

```
# ============================================================
# CONTINUAÇÃO DISCRETA DA LINHA DE CONTATO
# ============================================================


# ============================================================
# PRESSÃO DE CONTATO — V0.5
# ============================================================


# ============================================================
# CAMPO DE TENSÃO SUBSUPERFICIAL — V0.6
# ============================================================


# ============================================================
# TENSOR DE TENSÕES — V0.7
# ============================================================
```

```
def von_mises_from_tensor(t):
    """
    Von Mises para tensor cartesiano:

        sigma_VM =
        sqrt(
          0.5[
            (sx-sy)^2 +
            (sy-sz)^2 +
            (sz-sx)^2
            + 6(txy²+txz²+tyz²)
          ]
        )
    """
    sx = t["sx"]
    sy = t["sy"]
    sz = t["sz"]

    txy = t["txy"]
    txz = t["txz"]
    tyz = t["tyz"]

    value = 0.5 * (
        (sx - sy)**2
        + (sy - sz)**2
        + (sz - sx)**2
        + 6.0 * (
            txy*txy
            + txz*txz
            + tyz*tyz
        )
```

```
    )

    return math.sqrt(
        max(0.0, value)
    )


#
================================================================
===
# TENSOR ESPACIAL COMPLETO — V0.9
#
================================================================
===

def boussinesq_point_tensor(
    x,
    y,
    z,
    P,
    nu
):
    """
    Tensor cartesiano de Boussinesq para uma carga
pontual
    normal à superfície de um semi-espaço elástico
isotrópico.

    Convenção:
      z > 0: profundidade
      P > 0: magnitude da carga normal
      tensões normais compressivas: sinal negativo
```

Formulação cartesiana:

$$R = \sqrt{x^2+y^2+z^2}$$
$$r^2 = x^2+y^2$$

Para $r > 0$:

$$s_x = \frac{P}{2\pi}\left[\frac{1-2\nu}{r^2}\left(\frac{(1-z/R)(x^2-y^2)}{r^2} + \frac{z\,y^2}{R^3}\right) - \frac{3\,z\,x^2}{R^5}\right]$$

$$s_y = \frac{P}{2\pi}\left[\frac{1-2\nu}{r^2}\left(\frac{(1-z/R)(y^2-x^2)}{r^2} + \frac{z\,x^2}{R^3}\right) - \frac{3\,z\,y^2}{R^5}\right]$$

$$s_z = -\frac{3P\,z^3}{2\pi R^5}$$

$$t_{xy} = \frac{P}{2\pi}\left[\frac{1-2\nu}{r^2}\left(\frac{2xy(1-z/R)}{r^2} - \frac{xyz}{R^3}\right) - \frac{3xyz}{R^5}\right]$$

$$t_{xz} = -\frac{3P\,x\,z^2}{2\pi R^5}$$

$$t_{yz} = -\frac{3P\,y\,z^2}{2\pi R^5}$$

No eixo $r=0$ são usados os limites analíticos:

$$s_x = s_y = \frac{P(1-2\nu)}{4\pi z^2}$$

```
      sz = -3P/(2πz²)
      txy = txz = tyz = 0.

    Referência da formulação:
    Boussinesq para carga normal em semi-espaço elástico.
    """

    if z <= 0.0:
        raise ValidationError(
            "A profundidade z deve ser > 0."
        )

    if P < 0.0:
        raise ValidationError(
            "P deve ser >= 0 nesta convenção."
        )

    if not (0.0 <= nu < 0.5):
        raise ValidationError(
            "Poisson nu deve satisfazer 0 <= nu < 0.5."
        )

    R2 = x*x + y*y + z*z
    R = math.sqrt(R2)
    r2 = x*x + y*y

    c = P / (2.0 * math.pi)

    if r2 < 1.0e-30:
        sx = P * (1.0 - 2.0*nu) / (
            4.0 * math.pi * z*z
        )
        sy = sx
```

```
        sz = -3.0 * P / (
            2.0 * math.pi * z*z
        )

        return {
            "sx": sx,
            "sy": sy,
            "sz": sz,
            "txy": 0.0,
            "txz": 0.0,
            "tyz": 0.0,
        }

    R3 = R2 * R
    R5 = R3 * R2

    one_minus = 1.0 - 2.0*nu

    sx = c * (
        one_minus / r2 * (
            (1.0 - z/R) * (x*x - y*y)/r2
            + z*y*y/R3
        )
        - 3.0*z*x*x/R5
    )

    sy = c * (
        one_minus / r2 * (
            (1.0 - z/R) * (y*y - x*x)/r2
            + z*x*x/R3
        )
        - 3.0*z*y*y/R5
    )
```

```
    sz = -3.0 * c * z**3 / R5

    txy = c * (
        one_minus / r2 * (
            2.0*x*y*(1.0 - z/R)/r2
            - x*y*z/R3
        )
        - 3.0*x*y*z/R5
    )

    txz = -3.0 * c * x*z*z / R5
    tyz = -3.0 * c * y*z*z / R5

    return {
        "sx": sx,
        "sy": sy,
        "sz": sz,
        "txy": txy,
        "txz": txz,
        "tyz": tyz,
    }


def boussinesq_distributed_tensor(
    x0, y0, z, a, b, p0, nu, nx=40
):
    """
    Superposição numérica de Boussinesq em coordenadas
elípticas.

    r = sin(theta)
    x = a*r*cos(phi)
```

```
    y = b*r*sin(phi)

    dA = a*b*r*cos(theta) dtheta dphi
    p  = p0*cos(theta)

    Mesma soma de boussinesq_point_tensor (mesma ordem de
operações),
    com os invariantes do anel e o laço interno
inlinados: sem dict
    nem chamada de função por ponto.
    """
    if z <= 0.0:
        raise ValidationError("A profundidade z deve ser
> 0.")

    if not (0.0 <= nu < 0.5):
        raise ValidationError(
            "Poisson nu deve satisfazer 0 <= nu < 0.5."
        )

    ntheta = max(8, int(nx))
    if ntheta % 2:
        ntheta += 1
    nphi = 2 * ntheta

    dtheta = (math.pi / 2.0) / ntheta
    dphi = 2.0 * math.pi / nphi

    cos_phi = [math.cos((ip + 0.5) * dphi) for ip in
range(nphi)]
    sin_phi = [math.sin((ip + 0.5) * dphi) for ip in
range(nphi)]
```

```
    one_minus = 1.0 - 2.0*nu
    z3 = z**3
    z_3 = 3.0*z

    sx_t = sy_t = sz_t = 0.0
    txy_t = txz_t = tyz_t = 0.0

    for it in range(ntheta):
        theta = (it + 0.5) * dtheta
        r = math.sin(theta)
        c = math.cos(theta)
        p = p0 * c
        dA = a * b * r * c * dtheta * dphi
        dP = p * dA

        if dP < 0.0:
            raise ValidationError(
                "P deve ser >= 0 nesta convenção."
            )

        cc = dP / (2.0 * math.pi)
        m3c = -3.0 * cc
        a_r = a * r
        b_r = b * r

        for ip in range(nphi):
            x = a_r * cos_phi[ip] - x0
            y = b_r * sin_phi[ip] - y0

            r2 = x*x + y*y

            if r2 < 1.0e-30:
                sx = dP * one_minus / (
```

```
                        4.0 * math.pi * z*z
                    )
                    sx_t += sx
                    sy_t += sx
                    sz_t += -3.0 * dP / (
                        2.0 * math.pi * z*z
                    )
                    continue

                R2 = r2 + z*z
                R = math.sqrt(R2)
                R3 = R2 * R
                R5 = R3 * R2

                sx_t += cc * (
                    one_minus / r2 * (
                        (1.0 - z/R) * (x*x - y*y)/r2
                        + z*y*y/R3
                    )
                    - z_3*x*x/R5
                )

                sy_t += cc * (
                    one_minus / r2 * (
                        (1.0 - z/R) * (y*y - x*x)/r2
                        + z*x*x/R3
                    )
                    - z_3*y*y/R5
                )

                sz_t += m3c * z3 / R5

                txy_t += cc * (
```

```
                one_minus / r2 * (
                    2.0*x*y*(1.0 - z/R)/r2
                    - x*y*z/R3
                )
                - 3.0*x*y*z/R5
            )

            txz_t += m3c * x*z*z / R5
            tyz_t += m3c * y*z*z / R5

    return {
        "sx": sx_t,
        "sy": sy_t,
        "sz": sz_t,
        "txy": txy_t,
        "txz": txz_t,
        "tyz": tyz_t,
    }


def spatial_full_tensor_field(
    a,
    b,
    p0,
    nu,
    zmax,
    nx_xy,
    nz,
    integration_grid
):
    """
    Campo completo:
        sigma_x, sigma_y, sigma_z,
```

```
        tau_xy, tau_xz, tau_yz,
        sigma_VM

    em uma malha discreta (x,y,z).
    """

    nx_xy = max(3, int(nx_xy))
    nz = max(2, int(nz))

    field = []

    for ix in range(nx_xy):
        x0 = -a + 2.0*a*ix/(nx_xy - 1)

        for iy in range(nx_xy):
            y0 = -b + 2.0*b*iy/(nx_xy - 1)

            for iz in range(1, nz + 1):
                z = zmax * iz / nz

                tensor = boussinesq_distributed_tensor(
                    x0,
                    y0,
                    z,
                    a,
                    b,
                    p0,
                    nu,
                    integration_grid
                )

                vm = von_mises_from_tensor(
                    tensor
```

```
                )

                field.append({
                    "x": x0,
                    "y": y0,
                    "z": z,
                    "tensor": tensor,
                    "vm": vm,
                })

    return field


def full_tensor_extrema(field):
    if not field:
        return None, None

    vm_item = max(
        field,
        key=lambda q: q["vm"]
    )

    comp_item = min(
        field,
        key=lambda q: q["tensor"]["sz"]
    )

    return vm_item, comp_item


#
=============================================================
===
```

```
# CAMPO ESPACIAL SUBSUPERFICIAL — V0.8
#
================================================================
===



#
================================================================
===
# INTEGRIDADE — V1.0
#
================================================================
===



#
================================================================
===
# CRITÉRIO DE FADIGA — V1.1
#
================================================================
===



#
================================================================
===
# DANO ACUMULADO E VIDA — V1.2
#
================================================================
===

def sn_cycles_from_stress(
```

```
    sigma_a,
    sigma_ref,
    N_ref,
    exponent
):
    """
    Curva S-N de potência, explicitamente fornecida pelo
usuário:

        sigma_a / sigma_ref =
        (N / N_ref)^(-exponent)

    Portanto:

        N =
        N_ref *
        (sigma_a / sigma_ref)^(-1/exponent)

    Esta função não inventa uma curva S-N.
    Todos os parâmetros são entradas do caso.
    """
    if sigma_a <= 0:
        return float("inf")

    if sigma_ref <= 0:
        raise ValidationError(
            "sigma_ref deve ser > 0."
        )

    if N_ref <= 0:
        raise ValidationError(
            "N_ref deve ser > 0."
        )
```

```
    if exponent <= 0:
        raise ValidationError(
            "O expoente S-N deve ser > 0."
        )

    ratio = sigma_a / sigma_ref

    return (
        N_ref
        * ratio ** (-1.0 / exponent)
    )


def life_from_field(
    field,
    sigma_ref,
    N_ref,
    exponent
):
    """
    Para cada ponto do campo:

        sigma_a := sigma_VM

        N_f(x) = curva S-N
        D = N / N_f

    O ponto crítico é aquele com menor N_f.
    """
    result = []

    for item in field:
```

```
        sigma_a = item["vm"]

        N_f = sn_cycles_from_stress(
            sigma_a,
            sigma_ref,
            N_ref,
            exponent
        )

        result.append({
            "x": item["x"],
            "y": item["y"],
            "z": item["z"],
            "sigma_a": sigma_a,
            "N_f": N_f,
        })

    return result


def life_extrema(result):
    if not result:
        return None

    return min(
        result,
        key=lambda q: q["N_f"]
    )


#
============================================================
===
```

```
# V7.55 — E1 → E2 → CURVATURA RELATIVA → EHL
#
============================================================
===



#
============================================================
===
# V7.54 — CURVATURA RELATIVA NO PONTO DE CONTATO
#
============================================================
===



#
============================================================
===
# V7.53 — CURVATURAS PRINCIPAIS A PARTIR DE E1
#
============================================================
===



#
============================================================
===
# V7.52 — RAIOS DE CURVATURA EQUIVALENTES EHL
#
============================================================
===
```

```
#
============================================================
===
# V7.51 — E' REDUZIDO / EQUIVALENTE
#
============================================================
===



#
============================================================
===
# EHL — V1.3
#
============================================================
===

def ehl_dimensionless_groups(
    eta0,
    ue,
    Eprime,
    Rx,
    force,
    alpha_p
):
    if eta0 <= 0:
        raise ValidationError("eta0 deve ser > 0.")
    if ue < 0:
        raise ValidationError("u_e deve ser >= 0.")
    if Eprime <= 0:
        raise ValidationError("E' deve ser > 0.")
    if Rx <= 0:
        raise ValidationError("Rx deve ser > 0.")
```

```
    if force <= 0:
        raise ValidationError("F deve ser > 0.")
    if alpha_p < 0:
        raise ValidationError("alpha_p deve ser >= 0.")

    U = eta0 * ue / (Eprime * Rx)
    G = alpha_p * Eprime
    W = force / (Eprime * Rx * Rx)

    return U, G, W


def ehl_film_thickness(
    eta0,
    ue,
    Eprime,
    Rx,
    Ry,
    force,
    alpha_p
):
    """
    Correlações EHL elípticas já definidas no modelo:

      U = eta0*ue/(E'*Rx)
      G = alpha_p*E'
      W = F/(E'*Rx²)
      k = 1.03*(Ry/Rx)^0.64

      hmin/Rx =
      3.63 U^0.68 G^0.49 W^-0.073
      (1-exp(-0.68k))
```

```
      hc/Rx =
      2.69 U^0.67 G^0.53 W^-0.067
      (1-0.61 exp(-0.73k))
    """
    if Ry <= 0:
        raise ValidationError("Ry deve ser > 0.")

    U, G, W = ehl_dimensionless_groups(
        eta0,
        ue,
        Eprime,
        Rx,
        force,
        alpha_p
    )

    # Ellipticity parameter from the Hamrock-Dowson
point-contact correlation.
    k = 1.03 * (Ry / Rx) ** 0.64

    hmin = (
        Rx
        * 3.63
        * U**0.68
        * G**0.49
        * W**(-0.073)
        * (1.0 - math.exp(-0.68*k))
    )

    hc = (
        Rx
        * 2.69
        * U**0.67
```

```
        * G**0.53
        * W**(-0.067)
        * (1.0 - 0.61*math.exp(-0.73*k))
    )

    return {
        "U": U,
        "G": G,
        "W": W,
        "k": k,
        "hmin": hmin,
        "hc": hc,
    }


def film_parameter(
    hmin,
    Rq1,
    Rq2
):
    if hmin < 0:
        raise ValidationError(
            "hmin deve ser >= 0."
        )

    if Rq1 < 0 or Rq2 < 0:
        raise ValidationError(
            "Rq deve ser >= 0."
        )

    Rq = math.sqrt(
        Rq1*Rq1 + Rq2*Rq2
    )
```

```
    if Rq == 0:
        return float("inf"), Rq

    return hmin / Rq, Rq


#
================================================================
===
# ATRITO E TÉRMICA — V1.4
#
================================================================
===


def flash_temperature_rise(
    heat_flux,
    thermal_conductivity,
    thermal_diffusivity,
    contact_time
):
    """
    Modelo térmico local simplificado para elevação de
    temperatura por fonte superficial transitória:

        DeltaT_flash =
        q / k * sqrt(alpha * t / pi)

    Este módulo não substitui uma solução térmica
completa.
    """
    if heat_flux < 0:
```

```
        raise ValidationError("q deve ser >= 0.")
    if thermal_conductivity <= 0:
        raise ValidationError("k térmica deve ser > 0.")
    if thermal_diffusivity <= 0:
        raise ValidationError("alpha térmica deve ser >
0.")
    if contact_time < 0:
        raise ValidationError("tempo de contato deve ser
>= 0.")

    return (
        heat_flux
        / thermal_conductivity
        * math.sqrt(
            thermal_diffusivity
            * contact_time
            / math.pi
        )
    )


def bulk_temperature_rise(
    friction_power,
    thermal_conductance
):
    """
    Elevação de temperatura bulk em modelo concentrado:

        DeltaT_bulk = P_fric / H

    H é a condutância térmica equivalente do sistema.
    """
    if friction_power < 0:
```

```
        raise ValidationError("P_fric deve ser >= 0.")
    if thermal_conductance <= 0:
        raise ValidationError(
            "condutância térmica deve ser > 0."
        )

    return friction_power / thermal_conductance


def thermal_state(
    T0,
    heat_flux,
    thermal_conductivity,
    thermal_diffusivity,
    contact_time,
    friction_power,
    thermal_conductance
):
    dT_flash = flash_temperature_rise(
        heat_flux,
        thermal_conductivity,
        thermal_diffusivity,
        contact_time
    )

    dT_bulk = bulk_temperature_rise(
        friction_power,
        thermal_conductance
    )

    T_flash = T0 + dT_flash
    T_bulk = T0 + dT_bulk
    T_total = T0 + dT_bulk + dT_flash
```

```
    return {
        "dT_flash": dT_flash,
        "dT_bulk": dT_bulk,
        "T_flash": T_flash,
        "T_bulk": T_bulk,
        "T_total": T_total,
    }


#
=============================================================
===
# ACOPLAMENTO TÉRMICO-EHL — V1.5
#
=============================================================
===



#
=============================================================
===
# DESGASTE — V7.78 / CARGA DE ASPEREZAS EM EHL MISTO
#
=============================================================
===

def point_contact_asperity_load_ratio(
    dimensionless_load,
    dimensionless_speed,
    dimensionless_material,
    dimensionless_roughness,
    dimensionless_hardness,
```

```
):
    """
    Correlação de Masjedi & Khonsari para contato
pontual/elíptico:

        L_a = 10 W^(-0.083) U^(0.143) G^(0.314)
              ln[1 + σ̄^(4.689) V^(0.509)
                     W^(-0.501) U^(-2.90) G^(-2.870)]

    L_a é a razão de carga sustentada pelas asperezas,
expressa em %.

    Referência:
        Masjedi & Khonsari, Tribology International 82
(2015) 228–244,
        Eq. (23), DOI 10.1016/j.triboint.2014.09.010.
    """
    vals = [
        dimensionless_load,
        dimensionless_speed,
        dimensionless_material,
        dimensionless_roughness,
        dimensionless_hardness,
    ]
    if any(v <= 0 for v in vals):
        raise ValidationError(
            "W, U, G, sigma_bar e V devem ser > 0 para
calcular L_a."
        )

    term = (
        1.0
        + dimensionless_roughness ** 4.689
```

```
        * dimensionless_hardness ** 0.509
        * dimensionless_load ** (-0.501)
        * dimensionless_speed ** (-2.90)
        * dimensionless_material ** (-2.870)
    )

    La_percent = (
        10.0
        * dimensionless_load ** (-0.083)
        * dimensionless_speed ** 0.143
        * dimensionless_material ** 0.314
        * math.log(term)
    )

    if La_percent < 0:
        raise ValidationError("L_a calculado < 0%.")

    # A correlação fornece percentual; esta é a fração
usada no balanço de carga.
    La_fraction = La_percent / 100.0

    return {
        "La_percent": La_percent,
        "La_fraction": La_fraction,
    }


def mixed_ehl_archard_wear_volume(
    wear_coefficient,
    normal_force,
    asperity_load_fraction,
    sliding_distance,
    hardness,
```

```
):
    """
    Archard modificado para o desgaste associado ao
contato de asperezas:

        F_asp = L_a F_n
        V_w   = k_w F_asp s / H

    O coeficiente de desgaste K permanece o mesmo
parâmetro de entrada
    do V7.77; somente a força efetivamente aplicada à lei
de Archard
    passa a ser a parcela sustentada pelas asperezas.
    """
    if wear_coefficient < 0:
        raise ValidationError("k_w deve ser >= 0.")
    if normal_force < 0:
        raise ValidationError("F_n deve ser >= 0.")
    if not 0.0 <= asperity_load_fraction <= 1.0:
        raise ValidationError("A fração de carga de
asperezas deve estar entre 0 e 1.")
    if sliding_distance < 0:
        raise ValidationError("s deve ser >= 0.")
    if hardness <= 0:
        raise ValidationError("H deve ser > 0.")

    asperity_force = asperity_load_fraction *
normal_force
    volume = (
        wear_coefficient
        * asperity_force
        * sliding_distance
        / hardness
```

```
    )

    return {
        "asperity_force": asperity_force,
        "volume": volume,
    }


#
================================================================
===
# DESGASTE — V1.6
#
================================================================
===


def archard_wear_depth(
    wear_volume,
    contact_area
):
    """
    Profundidade média equivalente:

        d_w = V_w / A

    Não representa a distribuição espacial real do
desgaste.
    """
    if wear_volume < 0:
        raise ValidationError(
            "V_w deve ser >= 0."
        )
```

```
    if contact_area <= 0:
        raise ValidationError(
            "A deve ser > 0."
        )

    return wear_volume / contact_area


#
=============================================================
===
# INTEGRIDADE GLOBAL — V1.7
#
=============================================================
===



#
=============================================================
===
# INTEGRAÇÃO DOS RESULTADOS — V1.8
#
=============================================================
===



#
=============================================================
===
# PIPELINE AUTOMÁTICO — V1.9
#
=============================================================
===
```

```
#
# ============================================================
# PIPELINE AUTOMÁTICO DOS MÓDULOS — V7.19
#
# ============================================================



#
# ============================================================
# ESTADO DE LIMITES + INTEGRIDADE AUTOMÁTICA — V7.19
#
# ============================================================



#
# ============================================================
# ESTADO FINAL NORMALIZADO — V7.19
#
# ============================================================



#
# ============================================================
# VIDA LIMITE INTEGRADA — V7.19
```

```
#
============================================================
===



#
============================================================
===
# CÁLCULO AUTOMÁTICO DAS VIDAS — V7.19
#
============================================================
===



#
============================================================
===
# CONVERSÃO TEMPO ↔️ CICLOS — V7.19
#
============================================================
===

def time_to_cycles(time_value, cycle_frequency):
    """
    Converte tempo em ciclos somente quando a frequência
de ciclo
    é explicitamente fornecida:

        N = f t
    """
    if time_value < 0:
        raise ValidationError("tempo deve ser >= 0.")
    if cycle_frequency <= 0:
```

```
        raise ValidationError(
            "frequência de ciclo deve ser > 0."
        )
    return cycle_frequency * time_value


def cycles_to_time(cycles, cycle_frequency):
    """
        t = N / f
    """
    if cycles < 0:
        raise ValidationError("ciclos devem ser >= 0.")
    if cycle_frequency <= 0:
        raise ValidationError(
            "frequência de ciclo deve ser > 0."
        )
    return cycles / cycle_frequency


#
============================================================
===
# ESTADO GLOBAL DO MODELO — V7.19
#
============================================================
===



#
============================================================
===
# EVOLUÇÃO DISCRETA DO ESTADO — V7.19
#
```

```
# ============================================================
# AVALIAÇÃO FÍSICA POR ESTADO — V7.19
# ============================================================


# ============================================================
# INTEGRIDADE POR ESTADO E LIMITE DE VIDA — V7.19
# ============================================================


# ============================================================
# HISTÓRICO DE DANO E VIDA — V7.19
# ============================================================


# ============================================================
```

```
===
# ESTADO GLOBAL DE VIDA — V7.19
#
============================================================
===



#
============================================================
===
# RELATÓRIO DO ESTADO GLOBAL — V7.19
#
============================================================
===



#
============================================================
===
# INTEGRAÇÃO HERTZ → CAMPO 3D — V7.19
#
============================================================
===



#
============================================================
===
# HERTZ ELÍPTICO GERAL — V7.19
#
============================================================
===
```

```
#
================================================================
===
# SOLUÇÃO ELÁSTICA NUMÉRICA DO CONTATO — V7.19
#
================================================================
===


#
================================================================
===
# CONTATO ELÍPTICO EXPLÍCITO — V7.19
#
================================================================
===


#
================================================================
===
# PRESSÃO DE CONTATO DERIVADA — V7.19
#
================================================================
===


#
================================================================
===
# TENSÃO DERIVADA DO ESTADO DE CONTATO — V7.19
#
```

```
================================================================
===



#
================================================================
===
# CONTRATO DO ESTADO FÍSICO — V7.19
#
================================================================
===

PHYSICAL_STATE_REQUIRED = (
    "N", "t", "dt",
    "force", "Rx", "Ry", "ue", "v_slide", "Eprime",
"spherical_contact",
    "eta0", "alpha_p", "Rq1", "Rq2",
    "T0", "beta", "kth", "alpha_th",
    "contact_time", "H_bulk",
    "wear_coefficient", "hardness", "contact_area",
    "wear_limit_depth",
)



#
================================================================
===
# VERIFICAÇÃO E RASTREABILIDADE — V7.19
#
================================================================
===
```

```
#
============================================================
===
# INTERFACE MULTIPÁGINA + ROLAGEM
#
============================================================
===

class ScrollablePage(ttk.Frame):
    """
    Página vertical com Canvas + Scrollbar.
    Cada módulo futuro pode receber uma página própria.
    """

    def __init__(self, master):
        super().__init__(master)

        self.canvas = tk.Canvas(
            self,
            highlightthickness=0
        )
        self.scrollbar = ttk.Scrollbar(
            self,
            orient="vertical",
            command=self.canvas.yview
        )

        self.body = ttk.Frame(self.canvas)

        self.window_id = self.canvas.create_window(
            (0, 0),
            window=self.body,
            anchor="nw"
```

```
        )

        self.canvas.configure(
            yscrollcommand=self.scrollbar.set
        )

        self.canvas.pack(
            side="left",
            fill="both",
            expand=True
        )
        self.scrollbar.pack(
            side="right",
            fill="y"
        )

        self.body.bind(
            "<Configure>",
            self._update_scrollregion
        )
        self.canvas.bind(
            "<Configure>",
            self._resize_body
        )

        self.canvas.bind_all(
            "<MouseWheel>",
            self._mousewheel
        )

    def _update_scrollregion(self, event=None):
        self.canvas.configure(
            scrollregion=self.canvas.bbox("all")
```

```
        )

    def _resize_body(self, event):
        self.canvas.itemconfigure(
            self.window_id,
            width=event.width
        )

    def _mousewheel(self, event):
        try:
            self.canvas.yview_scroll(
                int(-event.delta / 120),
                "units"
            )
        except Exception:
            pass


class App:
    def __init__(self, root):
        self.root = root
        root.title("Modelo Tribológico Integrado — V7.78")

        # Pydroid 3 / Android: usar o tamanho real da tela e evitar uma
        # janela fixa maior que a área disponível. Não usar fullscreen, pois
        # em algumas versões do Tkinter no Android isso provoca tela preta.
        try:
            sw = int(root.winfo_screenwidth())
            sh = int(root.winfo_screenheight())
```

```
            w = max(360, min(sw, 900))
            h = max(520, min(sh, 1400))
            root.geometry(f"{w}x{h}+0+0")
            root.minsize(320, 480)
        except Exception:
            root.geometry("720x1200")
            root.minsize(320, 480)

        self.e = {}

        # ------------------------------------------------
-----
        # ESTILO
        # ------------------------------------------------
-----
        style = ttk.Style()
        try:
            style.theme_use("clam")
        except Exception:
            pass
        style.configure("App.TFrame",
background="#f3f5f7")
        style.configure("Header.TFrame",
background="#18222d")
        style.configure("Header.TLabel",
background="#18222d", foreground="white")
        style.configure("Title.TLabel",
background="#18222d", foreground="white", font=
("TkDefaultFont", 16, "bold"))
        style.configure("Sub.TLabel",
background="#18222d", foreground="#cbd5df", font=
("TkDefaultFont", 9))
        style.configure("Nav.TFrame",
```

```
background="#202b36")
        style.configure("Nav.TLabel",
background="#202b36", foreground="#dce4eb", font=
("TkDefaultFont", 9, "bold"))
        style.configure("Section.TLabel", font=
("TkDefaultFont", 12, "bold"))
        style.configure("Action.TButton", font=
("TkDefaultFont", 10, "bold"), padding=(12, 8))

        # Cabeçalho
        header = ttk.Frame(root, style="Header.TFrame",
padding=(14, 10))
        header.pack(fill="x")
        ttk.Label(header, text="MODELO TRIBOLÓGICO
INTEGRADO", style="Title.TLabel").pack(anchor="w")
        ttk.Label(header, text="V7.78  •  geometria →
contato → solicitações → integridade → vida",
style="Sub.TLabel").pack(anchor="w", pady=(2, 0))

        # Área principal: navegação fixa + página única
        main = ttk.Frame(root, style="App.TFrame")
        main.pack(fill="both", expand=True)

        nav = ttk.Frame(main, width=185,
style="Nav.TFrame", padding=(8, 8))
        nav.pack(side="left", fill="y")
        nav.pack_propagate(False)
        ttk.Label(nav, text="MÓDULOS",
style="Nav.TLabel").pack(anchor="w", pady=(2, 8))

        self.content = ttk.Frame(main,
style="App.TFrame", padding=(8, 8))
        self.content.pack(side="left", fill="both",
```

```
expand=True)

        # V7.82 — somente a página ENTRADA; o modelo
permanece no código.
        page_defs = [
            ("ENTRADA", "page_input"),
        ]
        self.page_defs = page_defs

        # Uma única página rolável por grupo. Os builders
matemáticos existentes
        # são reutilizados por alias para evitar
duplicação do núcleo de cálculo.
        for _, attr in page_defs:
            setattr(self, attr,
ScrollablePage(self.content))

        self.nav_list = tk.Listbox(
            nav, activestyle="none",
exportselection=False,
            bg="#202b36", fg="#dce4eb",
selectbackground="#3b82a0",
            selectforeground="white", relief="flat",
borderwidth=0,
            highlightthickness=0, font=("TkDefaultFont",
9),
        )
        for label, _ in page_defs:
            self.nav_list.insert("end", label)
        self.nav_list.pack(fill="both", expand=True)
        self.nav_list.bind("<<ListboxSelect>>",
self._select_page)
```

```
        footer = ttk.Frame(root, padding=(8, 4))
        footer.pack(fill="x")
        self.status_label = ttk.Label(footer,
text="Pronto")
        self.status_label.pack(side="left")
        ttk.Label(footer, text="V7.78 — interface
consolidada").pack(side="right")

        self.build_input_page()

        self.nav_list.selection_set(0)
        self.nav_list.event_generate("<<ListboxSelect>>")

    def _select_page(self, event=None):
        sel = self.nav_list.curselection()
        if not sel:
            return
        index = sel[0]
        for _, attr in self.page_defs:
            getattr(self, attr).pack_forget()
        label, attr = self.page_defs[index]
        page = getattr(self, attr)
        page.pack(fill="both", expand=True)
        self.status_label.configure(text=f"Módulo:
{label}")

    # ------------------------------------------------
-----
    # PÁGINA ENTRADA
    # ------------------------------------------------
-----

    def build_input_page(self):
```

```
        body = self.page_input.body

        ttk.Label(
            body,
            text="ENTRADAS GERAIS DO MODELO",
            font=("TkDefaultFont", 12, "bold")
        ).pack(anchor="w", pady=(8, 2))

        ttk.Label(
            body,
            text=(
                "Parâmetros que não pertencem à definição
geométrica E1. "
                "A geometria é configurada na página
GEOMETRIA + CONTATO."
            ),
            wraplength=430
        ).pack(anchor="w", pady=(0, 8))

        ttk.Label(
            body,
            text="PRESSÃO DE CONTATO — MODELO LOCAL",
            font=("TkDefaultFont", 12, "bold")
        ).pack(anchor="w", pady=(8, 2))

        pressure_fields = [
            ("force", "F normal [N]", "1"),
            ("a_contact", "semi-eixo a [m]", "0.001"),
            ("b_contact", "semi-eixo b [m]", "0.001"),
            ("pressure_grid", "grade pressão", "40"),
            ("nu_material", "Poisson ν [-]", "0.30"),
        ]
        self.add_fields(body, pressure_fields)
```

```
        ttk.Label(
            body,
            text="CAMPO SUBSUPERFICIAL",
            font=("TkDefaultFont", 12, "bold")
        ).pack(anchor="w", pady=(12, 2))

        stress_fields = [
            ("zmax", "profundidade máxima z [m]",
"0.005"),
            ("nz", "pontos em z", "30"),
            ("stress_grid", "grade integração", "40"),
        ]
        self.add_fields(body, stress_fields)

        # ------------------------------------------
-----
        # V7.77 — CASO FÍSICO INTEGRADO
        # Estas entradas são a fonte do caso executado
pela GUI.
        # Unidades de interface são convertidas para SI
antes do cálculo.
        # ------------------------------------------
-----
        ttk.Label(
            body,
            text="V7.78 — CASO FÍSICO: ENTRADAS
PRIMÁRIAS",
            font=("TkDefaultFont", 12, "bold")
        ).pack(anchor="w", pady=(14, 2))
        ttk.Label(
            body,
            text=(
```

```
                "Estes são os dados físicos que definem o
contato. "
                "R1 e R2 são raios dos dois corpos; não
são os semi-eixos Hertzianos a e b."
            ),
            wraplength=430
        ).pack(anchor="w", pady=(0, 6))

        self.add_fields(body, [
            ("v777_force", "F normal [N]", "1000"),
            ("v777_R1", "R1 [mm]", "1000000"),
            ("v777_R2", "R2 [mm]", "72.5"),
            ("v777_E1", "E1 [GPa]", "210"),
            ("v777_E2", "E2 [GPa]", "210"),
            ("v777_nu1", "ν1 [-]", "0.30"),
            ("v777_nu2", "ν2 [-]", "0.30"),
        ])

        ttk.Label(
            body,
            text="V7.78 — ENTRADAS TRIBOLÓGICAS /
TÉRMICAS / DESGASTE [ESTIMADAS]",
            font=("TkDefaultFont", 11, "bold")
        ).pack(anchor="w", pady=(12, 2))
        self.add_fields(body, [
            ("v777_ue", "velocidade média u_e [m/s]",
"15.5"),
            ("v777_vslide", "velocidade deslizamento
[m/s]", "8"),
            ("v777_eta0", "viscosidade η₀ [Pa.s]",
"0.12"),
            ("v777_alpha", "coef. pressão αp [Pa⁻¹]",
"2e-8"),
```

```
            ("v777_Rq1", "rugosidade Rq1 [µm]", "0.30"),
            ("v777_Rq2", "rugosidade Rq2 [µm]", "0.30"),
            ("v777_T0", "temperatura T₀ [°C]", "60"),
            ("v777_mu", "coeficiente µ [-]", "0.05"),
            ("v777_beta", "coef. térmico β [K⁻¹]",
"0.03"),
            ("v777_kcond", "condutividade [W/(m.K)]",
"40"),
            ("v777_diff", "difusividade [m²/s]", "1e-5"),
            ("v777_tcontact", "tempo de contato [s]",
"1e-3"),
            ("v777_hcond", "condutância [W/K]", "100"),
            ("v777_kwear", "coeficiente de desgaste K
[-]", "1e-8"),
            ("v777_H", "dureza H [GPa]", "7"),
            ("v777_tslide", "tempo de deslizamento [s]",
"3600"),
            ("v777_siglim", "limite σ [GPa]", "1.80"),
            ("v777_Tlim", "limite temperatura [°C]",
"180"),
            ("v777_hlim", "limite hmin [µm]", "0.50"),
            ("v777_wlim", "limite desgaste [µm]", "100"),
        ])

        ttk.Label(
            body,
            text="V7.78 — CICLOS / FADIGA / VIDA",
            font=("TkDefaultFont", 11, "bold")
        ).pack(anchor="w", pady=(12, 2))
        self.add_fields(body, [
            ("v777_freq", "frequência de ciclos [Hz]",
"100"),
            ("v777_N_eval", "ciclos para avaliação N
```

```
[-]", "100000"),
            ("v777_fat_sigref", "fadiga σ_ref [GPa]",
"1.80"),
            ("v777_fat_nref", "fadiga N_ref [ciclos]",
"1000000"),
            ("v777_fat_m", "expoente fadiga m [-]",
"0.12"),
        ])

        # Valores físicos fixados pelo caso solicitado.
        for _key in ("v777_R1", "v777_R2", "v777_ue",
"v777_vslide", "v777_freq"):
            self.e[_key].configure(state="disabled")

        ttk.Label(
            body,
            text="V7.79 — PITTING: CRITÉRIO CONVENCIONAL
DE RESISTÊNCIA SUPERFICIAL",
            font=("TkDefaultFont", 11, "bold")
        ).pack(anchor="w", pady=(12, 2))
        self.add_fields(body, [
            ("v779_pit_siglim", "σHlim pitting [GPa]",
"1.80"),
            ("v779_pit_shmin", "S_H mínimo [-]", "1.0"),
        ])

        ttk.Label(
            body,
            text="N de avaliação é o número de ciclos no
qual o estado acumulado será mostrado. A vida N_lim é
calculada pelo próprio modelo.",
            wraplength=430
        ).pack(anchor="w", pady=(0, 6))
```

```
        ttk.Label(
            body,
            text="V7.78 — RESULTADOS DERIVADOS",
            font=("TkDefaultFont", 11, "bold")
        ).pack(anchor="w", pady=(12, 2))
        self.v777_derived = tk.Text(body, height=8,
wrap="word", font=("Courier", 9))
        self.v777_derived.pack(fill="x", pady=(2, 6))

        ttk.Button(
            body,
            text="EXECUTAR CASO FÍSICO V7.78",
            command=self.run_v777_gui,
            style="Action.TButton"
        ).pack(fill="x", pady=(2, 8))

        self.v777_report = tk.Text(body, height=20,
wrap="word", font=("Courier", 9))
        self.v777_report.pack(fill="x", pady=(2, 10))

        self.input_status = tk.Text(
            body,
            height=7,
            wrap="word"
        )
        self.input_status.pack(
            fill="x",
            pady=10
        )

    def run_v777_gui(self):
        """Lê todas as entradas V7.78 da GUI, converte
```

```
para SI e executa o caso."""
        try:
            def f(key):
                return
float(self.e[key].get().strip().replace(",", "."))

            case = dict(V776_BASE_CASE)
            case.update({
                "force": f("v777_force"),
                "R1": 1.0e3,                # FIXO: 1e6
mm
                "R2": 72.5e-3,              # FIXO: 72.5
mm
                "E1": f("v777_E1") * 1e9,
                "E2": f("v777_E2") * 1e9,
                "nu1": f("v777_nu1"),
                "nu2": f("v777_nu2"),
                "ue": 15.5,                 # FIXO: m/s
                "v_slide": 8.0,              # FIXO: m/s
                "eta0": f("v777_eta0"),
                "alpha_p": f("v777_alpha"),
                "Rq1": f("v777_Rq1") * 1e-6,
                "Rq2": f("v777_Rq2") * 1e-6,
                "T0": f("v777_T0") + 273.15,
                "mu": f("v777_mu"),
                "beta": f("v777_beta"),
                "thermal_conductivity": f("v777_kcond"),
                "thermal_diffusivity": f("v777_diff"),
                "contact_time": f("v777_tcontact"),
                "thermal_conductance": f("v777_hcond"),
                "wear_coefficient": f("v777_kwear"),
                "hardness": f("v777_H") * 1e9,
                "sliding_time": f("v777_tslide"),
```

```
                "sigma_limit": f("v777_siglim") * 1e9,
                "temperature_limit": f("v777_Tlim") +
273.15,
                "hmin_limit": f("v777_hlim") * 1e-6,
                "wear_limit_depth": f("v777_wlim") * 1e-
6,
                "cycle_frequency": 100.0,     # FIXO: Hz
                "cycles_evaluation": f("v777_N_eval"),
                "fatigue_sigma_ref": f("v777_fat_sigref")
* 1e9,
                "fatigue_N_ref": f("v777_fat_nref"),
                "fatigue_exponent_m": f("v777_fat_m"),
                "pitting_sigma_Hlim":
f("v779_pit_siglim") * 1e9,
                "pitting_SH_min": f("v779_pit_shmin"),
            })

            result = execute_v776_reference_case(case)
            eq = result["equivalent"]
            self.v777_derived.delete("1.0", "end")
            self.v777_derived.insert("1.0",
                "E'  = %.8e Pa  (%.8f GPa)\n"
                "R'  = %.8e m   (%.8f mm)\n"
                "a=b = %.8e m   (%.8f mm)\n"
                "A   = %.8e m²\n"
                "pH  = %.8e Pa\n"
                "pm  = %.8e Pa" % (
                    eq["Eprime"], eq["Eprime"] / 1e9,
                    eq["Rprime"], eq["Rprime"] * 1e3,
                    eq["a"], eq["a"] * 1e3,
                    eq["contact_area"], eq["p0"],
eq["pm"]
                )
```

```
            )
            self.v777_report.delete("1.0", "end")
            self.v777_report.insert("1.0",
format_v776_reference_report(result).replace("V7.77",
"V7.77"))
            self.status_label.configure(text="V7.78 —
caso físico calculado")

        except Exception as exc:
            self.v777_derived.delete("1.0", "end")
            self.v777_report.delete("1.0", "end")
            self.v777_report.insert("1.0", "ERRO
V7.78\n\n" + str(exc))
            self.status_label.configure(text="V7.78 —
erro de entrada/cálculo")

    def add_fields(self, body, fields):
        for key, label, default in fields:
            row = ttk.Frame(body)
            row.pack(
                fill="x",
                pady=2
            )

            ttk.Label(
                row,
                text=label,
                width=25
            ).pack(side="left")

            ent = ttk.Entry(row)
            ent.pack(
                side="right",
```

```
                fill="x",
                expand=True
            )
            ent.insert(0, default)

            self.e[key] = ent

    # ---------------------------------------------------
-----
    # PÁGINA GEOMETRIA
    # ---------------------------------------------------
-----



    # ---------------------------------------------------
-----
    # PÁGINA CONTATO
    # ---------------------------------------------------
-----



    # ---------------------------------------------------
-----
    # PÁGINA PRESSÃO
    # ---------------------------------------------------
-----



    # ---------------------------------------------------
-----
    # PÁGINA TENSÃO
    # ---------------------------------------------------
-----
```

```
    # ------------------------------------------------------
-----
    # PÁGINA TENSOR
    # ------------------------------------------------------
-----


    # ------------------------------------------------------
-----
    # PÁGINA CAMPO ESPACIAL
    # ------------------------------------------------------
-----


    # ------------------------------------------------------
-----
    # PÁGINA INTEGRIDADE
    # ------------------------------------------------------
-----


    # ------------------------------------------------------
-----
    # PÁGINA FADIGA
    # ------------------------------------------------------
-----


    # ------------------------------------------------------
-----
    # PÁGINA VIDA
```

```
    # ------------------------------------------------------
-----



    # ------------------------------------------------------
-----
    # PÁGINA EHL
    # ------------------------------------------------------
-----



    # ------------------------------------------------------
-----
    # PÁGINA TÉRMICA
    # ------------------------------------------------------
-----



    # ------------------------------------------------------
-----
    # PÁGINA ACOPLAMENTO TÉRMICO-EHL
    # ------------------------------------------------------
-----



    # ------------------------------------------------------
-----
    # PÁGINA DESGASTE
    # ------------------------------------------------------
-----



    # ------------------------------------------------------
```

```
-----
    # PÁGINA INTEGRIDADE GLOBAL
    # ----------------------------------------------------
-----



    # ----------------------------------------------------
-----
    # PÁGINA INTEGRAÇÃO
    # ----------------------------------------------------
-----



    # ----------------------------------------------------
-----
    # PÁGINA PIPELINE
    # ----------------------------------------------------
-----



    # ----------------------------------------------------
-----
    # PÁGINA PIPELINE AUTOMÁTICO
    # ----------------------------------------------------
-----



    # ----------------------------------------------------
-----
    # PÁGINA VOLUME
    # ----------------------------------------------------
-----
```

```
    # ------------------------------------------------
-----
    # EXECUÇÃO
    # ------------------------------------------------
-----


    # ------------------------------------------------
-----
    # SAÍDAS
    # ------------------------------------------------
-----


#
================================================
===
# V7.62 — CONSTRUCAO DO CAMPO COMUM A PARTIR DE E2 + E4 +
EHL
#
================================================
===


if False and __name__ == "__main__":
    root = tk.Tk()
    App(root)
    root.mainloop()


# ========================= V7.66 — FADIGA MECÂNICA
EFETIVA ========================
```

```
# ========================= V7.67 — INTEGRIDADE EVOLUTIVA
UNIFICADA =========================


# ========================= V7.68 — EVOLUÇÃO COMPLETA
TEMPO/CICLOS =========================


# ========================= V7.69 — FECHAMENTO DO
PIPELINE E1→E5 + VIDA =========================


#
============================================================
===
# V7.70 — AUDITORIA FÍSICO-MATEMÁTICA DO CASO FECHADO
#
============================================================
===


#
============================================================
===
# V7.77 — FECHAMENTO DAS LACUNAS FÍSICO-MATEMÁTICAS
#
============================================================
===


#
```

```
# ============================================================
# V7.77 — CASO FÍSICO DE REFERÊNCIA
#
# ============================================================
#
# Esta extensão não substitui os módulos anteriores.
# Ela estabelece um caso físico central e deriva os resultados
# a partir de entradas físicas primárias.
#
# Entradas centrais:
#     F  = 1000 N
#     R1 = 25 mm
#     R2 = 25 mm
#     E1 = E2 = 210 GPa
#     nu1 = nu2 = 0.30
#
# O raio relativo é DERIVADO:
#     1/R' = 1/R1 + 1/R2
#
# Entradas tribológicas não determinadas por Hertz permanecem
# explicitamente classificadas como ESTIMATIVAS.
#
# ============================================================

V777_VERSION = "V7.78"
V779_VERSION = "V7.79"
V780_VERSION = "V7.81"
```

```
V776_BASE_CASE = {
    # ------------------------
    # Entradas físicas primárias
    # ------------------------
    "force": 1000.0,                # N
    "R1": 1.0e3,                     # m (1e6 mm =
infinito numerico)
    "R2": 72.5e-3,                   # m (72.5 mm)
    "E1": 210.0e9,                  # Pa
    "E2": 210.0e9,                  # Pa
    "nu1": 0.30,
    "nu2": 0.30,

    # ------------------------
    # Entradas tribológicas estimadas
    # ------------------------
    "ue": 15.5,                     # m/s
    "v_slide": 8.0,                  # m/s
    "eta0": 0.12,                   # Pa.s
    "alpha_p": 2.0e-8,              # Pa^-1
    "Rq1": 0.30e-6,                 # m
    "Rq2": 0.30e-6,                 # m
    "T0": 60.0 + 273.15,            # K
    "mu": 0.05,

    # ------------------------
    # Parâmetros térmicos estimados
    # ------------------------
    "beta": 0.03,                   # K^-1
    "thermal_conductivity": 40.0,   # W/(m.K)
    "thermal_diffusivity": 1.0e-5,  # m²/s
    "contact_time": 1.0e-3,         # s
    "thermal_conductance": 100.0,   # W/K
```

```
    # ------------------------
    # Desgaste estimado
    # ------------------------
    "wear_coefficient": 1.0e-8,
    "hardness": 7.0e9,               # Pa
    "sliding_time": 3600.0,          # s
    "cycle_frequency": 100.0,        # Hz
    "cycles_evaluation": 100000.0,   # ciclos

    # ------------------------
    # Limites de referência estimados
    # ------------------------
    "sigma_limit": 1.80e9,           # Pa
    "fatigue_sigma_ref": 1.80e9,     # Pa
    "fatigue_N_ref": 1.0e6,          # ciclos
    "fatigue_exponent_m": 0.12,

    # ------------------------
    # Pitting — V7.79
    # Criterio convencional de resistência superficial de
worm:
    # S_H = sigma_HG / sigma_Hm ; S_H,min = 1.0.
    # sigma_HG deve ser fornecido para o
material/condição reais.
    # ------------------------
    "pitting_sigma_Hlim": 1.80e9,   # Pa; referência de
contato já usada no caso-base
    "pitting_SH_min": 1.0,           # fator mínimo de
segurança
    "temperature_limit": 180.0 + 273.15,
    "hmin_limit": 0.50e-6,
    "wear_limit_depth": 100.0e-6,    # m
```

```
}


def derive_v776_equivalent_properties(case):
    """Deriva E' e R' a partir dos dois corpos do caso-
base."""
    E1 = float(case["E1"])
    E2 = float(case["E2"])
    nu1 = float(case["nu1"])
    nu2 = float(case["nu2"])
    R1 = float(case["R1"])
    R2 = float(case["R2"])

    if E1 <= 0 or E2 <= 0:
        raise ValidationError("E1 e E2 devem ser > 0.")
    if not (0.0 <= nu1 < 0.5 and 0.0 <= nu2 < 0.5):
        raise ValidationError("nu1 e nu2 devem estar em 0
<= nu < 0.5.")
    if R1 <= 0 or R2 <= 0:
        raise ValidationError("R1 e R2 devem ser > 0.")

    Eprime = 1.0 / (
        (1.0 - nu1**2) / E1
        + (1.0 - nu2**2) / E2
    )
    Rprime = 1.0 / (1.0 / R1 + 1.0 / R2)

    return {
        "Eprime": Eprime,
        "Rprime": Rprime,
    }
```

```
def derive_v776_hertz(case):
    """Calcula contato Hertziano circular a partir de F e
R'."""
    eq = derive_v776_equivalent_properties(case)
    F = float(case["force"])
    Eprime = eq["Eprime"]
    Rprime = eq["Rprime"]

    if F < 0:
        raise ValidationError("F deve ser >= 0.")

    a = ((3.0 * F * Rprime) / (4.0 * Eprime)) ** (1.0 /
3.0)
    b = a
    area = math.pi * a * b
    p0 = 3.0 * F / (2.0 * area)
    pm = F / area

    return {
        **eq,
        "a": a,
        "b": b,
        "contact_area": area,
        "p0": p0,
        "pm": pm,
    }


def derive_v781_pitting(c, hertz):
    """Pitting V7.79: S_H = sigma_HG / sigma_Hm, sem
inventar sigma_HG."""
    pitting_sigma_Hm = float(hertz["pm"])
    pitting_sigma_Hlim = c.get("pitting_sigma_Hlim")
```

```
    pitting_SH_min = float(c.get("pitting_SH_min", 1.0))
    if pitting_sigma_Hlim is None:
        pitting = {
            "enabled": False,
            "criterion": "S_H = sigma_HG / sigma_Hm",
            "sigma_Hm": pitting_sigma_Hm,
            "sigma_Hlim": None,
            "S_H": None,
            "S_H_min": pitting_SH_min,
            "U_pitting": None,
            "admissible": None,
            "reason": "sigma_Hlim de pitting nao
informado para o material real",
        }
    else:
        pitting_sigma_Hlim = float(pitting_sigma_Hlim)
        if pitting_sigma_Hlim <= 0.0:
            raise ValidationError("pitting_sigma_Hlim
deve ser > 0 quando informado.")
        if pitting_SH_min <= 0.0:
            raise ValidationError("pitting_SH_min deve
ser > 0.")
        pitting_SH = pitting_sigma_Hlim /
pitting_sigma_Hm if pitting_sigma_Hm > 0.0 else
float("inf")
        pitting_U = pitting_SH_min / pitting_SH if
pitting_SH > 0.0 else float("inf")
        pitting = {
            "enabled": True,
            "criterion": "S_H = sigma_HG / sigma_Hm",
            "sigma_Hm": pitting_sigma_Hm,
            "sigma_Hlim": pitting_sigma_Hlim,
            "S_H": pitting_SH,
```

```
            "S_H_min": pitting_SH_min,
            "U_pitting": pitting_U,
            "admissible": pitting_SH >= pitting_SH_min,
            "contact_pressure_basis": "pm (Hertz mean
contact stress)",
            "load_factor_to_limit": (pitting_SH /
pitting_SH_min) ** 3,
        }
    return pitting


def derive_v781_surface_capacity(c, pitting, ehl, Lambda,
Rq):
    """Capacidade de superfície V7.81: margens macro e
micropitting."""
    macro_capacity = {
        "enabled": bool(pitting.get("enabled", False)),
        "sigma_Hm": pitting.get("sigma_Hm"),
        "sigma_Hlim": pitting.get("sigma_Hlim"),
        "S_H": pitting.get("S_H"),
        "S_H_min": pitting.get("S_H_min"),
        "load_factor_to_limit": None,
        "force_limit": None,
    }
    if macro_capacity["enabled"] and
macro_capacity["S_H"] is not None:
        SH = float(macro_capacity["S_H"])
        SHmin = float(macro_capacity["S_H_min"])
        if SH > 0.0 and SHmin > 0.0:
            # Exato para o Hertz circular atualmente
implementado:
            # p_m proporcional a F^(1/3).
            macro_capacity["load_factor_to_limit"] = (SH
```

```
/ SHmin) ** 3
            macro_capacity["force_limit"] =
float(c["force"]) *
macro_capacity["load_factor_to_limit"]

    lambda_current = float(Lambda)
    rq_current = float(Rq)
    hmin_limit = float(c["hmin_limit"])
    lambda_min = hmin_limit / rq_current if rq_current >
0.0 else float("inf")

    micro_capacity = {
        "criterion": "Lambda = h_min / sqrt(Rq1^2 +
Rq2^2)",
        "h_min": float(ehl["hmin"]),
        "h_min_limit": hmin_limit,
        "Rq": rq_current,
        "Lambda": lambda_current,
        "Lambda_min": lambda_min,
        "S_lambda": (lambda_current / lambda_min) if
lambda_min > 0.0 and math.isfinite(lambda_min) else
float("inf"),
        "admissible": lambda_current >= lambda_min,
        "load_factor_to_limit": None,
        "force_limit": None,
        "life_predictable": False,
        "time_to_first_micropitting": None,
        "reason_no_time_prediction": "Lambda define
margem de filme; nao fornece lei de nucleacao de
micropitting em ciclos.",
    }
    if lambda_current > 0.0 and lambda_min > 0.0 and
math.isfinite(lambda_min):
```

```
        # h_min = C * F^(-0.073), demais entradas
mantidas constantes.
        micro_capacity["load_factor_to_limit"] =
(lambda_current / lambda_min) ** (1.0 / 0.073)
        micro_capacity["force_limit"] = float(c["force"])
* micro_capacity["load_factor_to_limit"]
    return macro_capacity, micro_capacity


def derive_v781_friction_thermal(c, hertz, ehl):
    """Atrito derivado de F, mu e v_slide; térmica a
partir do cisalhamento do filme."""
    friction_force = float(c["mu"]) * float(c["force"])
    friction_power = friction_force *
abs(float(c["v_slide"]))
    tau_lub = (
        float(c["eta0"])
        * abs(float(c["v_slide"]))
        / ehl["hmin"]
    )
    q_fric = tau_lub * abs(float(c["v_slide"]))
    p_fric_ehl = q_fric * hertz["contact_area"]

    # Térmica: fluxo derivado do cisalhamento do filme;
potência global
    # também é reportada separadamente pela aproximação
de Coulomb.
    thermal = thermal_state(
        T0=float(c["T0"]),
        heat_flux=q_fric,

thermal_conductivity=float(c["thermal_conductivity"]),
```

```
thermal_diffusivity=float(c["thermal_diffusivity"]),
        contact_time=float(c["contact_time"]),
        friction_power=p_fric_ehl,

thermal_conductance=float(c["thermal_conductance"]),
    )
    return friction_force, friction_power, tau_lub,
q_fric, p_fric_ehl, thermal


def derive_v781_wear(c, hertz, ehl, Rq):
    """Desgaste V7.78: Archard sobre a parcela de carga
das asperezas."""
    sigma_bar = Rq / hertz["Rprime"]
    V_dimless = float(c["hardness"]) / hertz["Eprime"]
    asperity = point_contact_asperity_load_ratio(
        ehl["W"],
        ehl["U"],
        ehl["G"],
        sigma_bar,
        V_dimless,
    )
    mixed_wear = mixed_ehl_archard_wear_volume(
        float(c["wear_coefficient"]),
        float(c["force"]),
        asperity["La_fraction"],
        abs(float(c["v_slide"])) *
float(c["sliding_time"]),
        float(c["hardness"]),
    )
    sliding_distance = abs(float(c["v_slide"])) *
float(c["sliding_time"])
    wear_volume = mixed_wear["volume"]
```

```
    wear_depth = archard_wear_depth(
        wear_volume,
        hertz["contact_area"],
    )
    return asperity, mixed_wear, sliding_distance,
wear_volume, wear_depth


def derive_v781_life_limits(c, stress_field, wear_depth):
    """Vida por fadiga (S-N do caso) e por desgaste, em
ciclos e tempo."""
    fatigue_field = life_from_field(
        stress_field,
        float(c["fatigue_sigma_ref"]),
        float(c["fatigue_N_ref"]),
        float(c["fatigue_exponent_m"]),
    )
    fatigue_limit = life_extrema(fatigue_field)
    N_fatigue = (
        float(fatigue_limit["N_f"])
        if fatigue_limit is not None else float("inf")
    )

    frequency = float(c.get("cycle_frequency", 0.0))
    if frequency <= 0:
        raise ValidationError("A frequência de ciclos
deve ser > 0 para calcular vida em ciclos.")

    t_fatigue = cycles_to_time(N_fatigue, frequency) if
math.isfinite(N_fatigue) else float("inf")

    # O desgaste calculado acima corresponde ao tempo de
referência
```

```
    # sliding_time. Sob carga e velocidade constantes, a
taxa média é constante.
    t_slide_ref = float(c["sliding_time"])
    wear_depth_rate = wear_depth / t_slide_ref if
t_slide_ref > 0 else 0.0
    t_wear = (
        float(c["wear_limit_depth"]) / wear_depth_rate
        if wear_depth_rate > 0 else float("inf")
    )
    N_wear = time_to_cycles(t_wear, frequency) if
math.isfinite(t_wear) else float("inf")
    return N_fatigue, frequency, t_fatigue,
wear_depth_rate, t_wear, N_wear


def derive_v781_instant_integrity(c, vm_max, ehl, Lambda,
thermal, wear_depth):
    """Integridade imediata no instante do caso-base."""
    sigma_vm = float(vm_max["vm"])
    G_sigma = float(c["sigma_limit"]) - sigma_vm
    U_sigma = sigma_vm / float(c["sigma_limit"])
    U_hmin = float(c["hmin_limit"]) / ehl["hmin"]
    U_lambda = 1.0 / Lambda if Lambda > 0 else
float("inf")
    U_temperature = thermal["T_total"] /
float(c["temperature_limit"])
    U_wear = wear_depth / float(c["wear_limit_depth"])
    U_global = max(U_sigma, U_lambda, U_temperature,
U_wear)
    return sigma_vm, G_sigma, U_sigma, U_hmin, U_lambda,
U_temperature, U_wear, U_global
```

```
def derive_v781_limiting_life(U_global, N_fatigue,
t_fatigue, N_wear, t_wear):
    """Vida global: zero se U>=1; senão a primeira falha
acumulada."""
    if U_global >= 1.0:
        N_limit = 0.0
        t_limit = 0.0
        limiting_mechanism = "integridade instantanea"
    elif N_fatigue <= N_wear:
        N_limit = N_fatigue
        t_limit = t_fatigue
        limiting_mechanism = "fadiga"
    else:
        N_limit = N_wear
        t_limit = t_wear
        limiting_mechanism = "desgaste"
    return N_limit, t_limit, limiting_mechanism


def derive_v781_evaluation_state(
    c, frequency, N_fatigue, wear_depth_rate,
    U_sigma, U_lambda, U_temperature
):
    """Estado acumulado no número de ciclos de avaliação
do usuário."""
    N_eval = float(c.get("cycles_evaluation", 0.0))
    if N_eval < 0:
        raise ValidationError("Os ciclos de avaliação
devem ser >= 0.")
    t_eval = cycles_to_time(N_eval, frequency)
    D_fatigue_eval = (N_eval / N_fatigue) if
math.isfinite(N_fatigue) else 0.0
    wear_depth_eval = wear_depth_rate * t_eval
```

```
    U_wear_eval = wear_depth_eval /
float(c["wear_limit_depth"])
    U_global_eval = max(
        U_sigma,
        U_lambda,
        U_temperature,
        U_wear_eval,
        D_fatigue_eval,
    )
    return N_eval, t_eval, D_fatigue_eval, U_global_eval


def execute_v776_reference_case(case=None):
    """Executa o caso-base V7.77 sem substituir entradas
ausentes por valores."""
    c = dict(V776_BASE_CASE if case is None else case)
    hertz = derive_v776_hertz(c)
    pitting = derive_v781_pitting(c, hertz)

    # Campo espacial de tensões: malha moderada para o
relatório-base.
    stress_field = spatial_full_tensor_field(
        a=hertz["a"],
        b=hertz["b"],
        p0=hertz["p0"],
        nu=float(c["nu1"]),
        zmax=3.0 * hertz["a"],
        nx_xy=9,
        nz=12,
        integration_grid=32,
    )
    vm_max, sz_min = full_tensor_extrema(stress_field)
```

```
    # EHL: Rx/Ry são os raios relativos derivados do
mesmo contato.
    ehl = ehl_film_thickness(
        float(c["eta0"]),
        float(c["ue"]),
        hertz["Eprime"],
        hertz["Rprime"],
        hertz["Rprime"],
        float(c["force"]),
        float(c["alpha_p"]),
    )
    Lambda, Rq = film_parameter(
        ehl["hmin"],
        float(c["Rq1"]),
        float(c["Rq2"]),
    )

    macro_capacity, micro_capacity =
derive_v781_surface_capacity(
        c, pitting, ehl, Lambda, Rq
    )
    (
        friction_force, friction_power, tau_lub, q_fric,
        p_fric_ehl, thermal
    ) = derive_v781_friction_thermal(c, hertz, ehl)
    (
        asperity, mixed_wear, sliding_distance,
        wear_volume, wear_depth
    ) = derive_v781_wear(c, hertz, ehl, Rq)
    (
        N_fatigue, frequency, t_fatigue,
        wear_depth_rate, t_wear, N_wear
    ) = derive_v781_life_limits(c, stress_field,
```

```
wear_depth)
    (
        sigma_vm, G_sigma, U_sigma, U_hmin,
        U_lambda, U_temperature, U_wear, U_global
    ) = derive_v781_instant_integrity(
        c, vm_max, ehl, Lambda, thermal, wear_depth
    )
    N_limit, t_limit, limiting_mechanism =
derive_v781_limiting_life(
        U_global, N_fatigue, t_fatigue, N_wear, t_wear
    )
    N_eval, t_eval, D_fatigue_eval, U_global_eval =
derive_v781_evaluation_state(
        c, frequency, N_fatigue, wear_depth_rate,
        U_sigma, U_lambda, U_temperature
    )

    return {
        "version": V780_VERSION,
        "inputs": c,
        "equivalent": hertz,
        "hertz": hertz,
        "stress": {
            "field_points": len(stress_field),
            "sigma_VM_max": sigma_vm,
            "sigma_VM_point": {
                "x": vm_max["x"], "y": vm_max["y"], "z":
vm_max["z"]
            },
            "sigma_z_min": sz_min["tensor"]["sz"],
            "sigma_z_point": {
                "x": sz_min["x"], "y": sz_min["y"], "z":
sz_min["z"]
```

```
            },
        },
        "ehl": {
            **ehl,
            "Rq": Rq,
            "Lambda": Lambda,
        },
        "friction": {
            "friction_force": friction_force,
            "friction_power_coulomb": friction_power,
            "tau_lub": tau_lub,
            "q_fric": q_fric,
            "P_fric_ehl": p_fric_ehl,
        },
        "thermal": thermal,
        "pitting": pitting,
        "surface_capacity": {
            "macro": macro_capacity,
            "micro": micro_capacity,
        },
        "wear": {
            "sliding_distance": sliding_distance,
            "normal_force": float(c["force"]),
            "asperity_load_ratio_percent":
asperity["La_percent"],
            "asperity_load_fraction":
asperity["La_fraction"],
            "asperity_force":
mixed_wear["asperity_force"],
            "volume": wear_volume,
            "depth": wear_depth,
        },
        "integrity": {
```

```
            "G_sigma": G_sigma,
            "U_sigma": U_sigma,
            "U_hmin": U_hmin,
            "U_lambda": U_lambda,
            "U_temperature": U_temperature,
            "U_wear": U_wear,
            "U_global": U_global,
            "U_global_eval": U_global_eval,
        },
        "fatigue": {
            "sigma_ref": float(c["fatigue_sigma_ref"]),
            "N_ref": float(c["fatigue_N_ref"]),
            "exponent_m": float(c["fatigue_exponent_m"]),
            "N_f": N_fatigue,
            "t_f": t_fatigue,
            "D_eval": D_fatigue_eval,
        },
        "life": {
            "frequency": frequency,
            "N_eval": N_eval,
            "t_eval": t_eval,
            "wear_depth_rate": wear_depth_rate,
            "N_wear": N_wear,
            "t_wear": t_wear,
            "N_fatigue": N_fatigue,
            "t_fatigue": t_fatigue,
            "N_lim": N_limit,
            "t_lim": t_limit,
            "limiting_mechanism": limiting_mechanism,
        },
        "status": "calculado",
    }
```

```
def _v776_fmt(x):
    return f"{float(x):.8e}"


def format_v776_reference_report(result):
    """Relatório integral em texto puro, sem perda de
dados numéricos."""
    c = result["inputs"]
    h = result["hertz"]
    s = result["stress"]
    e = result["ehl"]
    f = result["friction"]
    t = result["thermal"]
    w = result["wear"]
    g = result["integrity"]
    cap = result["surface_capacity"]
    macro = cap["macro"]
    micro = cap["micro"]

    lines = [
        "=== MODELO TRIBOLÓGICO INTEGRADO V7.81 ===",
        "=== CASO FÍSICO DE REFERÊNCIA ===",
        "",
        "ENTRADAS PRIMÁRIAS",
        f"F                 = {_v776_fmt(c['force'])}
N",
        f"R1                = {_v776_fmt(c['R1'])} m",
        f"R2                = {_v776_fmt(c['R2'])} m",
        f"E1                = {_v776_fmt(c['E1'])} Pa",
        f"E2                = {_v776_fmt(c['E2'])} Pa",
        f"nu1               = {_v776_fmt(c['nu1'])}",
        f"nu2               = {_v776_fmt(c['nu2'])}",
```

```
        "",
        "PROPRIEDADES EQUIVALENTES",
        f"E'                = {_v776_fmt(h['Eprime'])}
Pa",
        f"R'                = {_v776_fmt(h['Rprime'])}
m",
        "",
        "CONTATO HERTZIANO",
        f"a                 = {_v776_fmt(h['a'])} m",
        f"b                 = {_v776_fmt(h['b'])} m",
        f"area              =
{_v776_fmt(h['contact_area'])} m2",
        f"p_media           = {_v776_fmt(h['pm'])} Pa",
        f"p_H_max           = {_v776_fmt(h['p0'])} Pa",
        "",
        "CAMPO DE TENSOES",
        f"pontos            = {s['field_points']}",
        f"sigma_VM_max      =
{_v776_fmt(s['sigma_VM_max'])} Pa",
        f"VM_x              =
{_v776_fmt(s['sigma_VM_point']['x'])} m",
        f"VM_y              =
{_v776_fmt(s['sigma_VM_point']['y'])} m",
        f"VM_z              =
{_v776_fmt(s['sigma_VM_point']['z'])} m",
        f"sigma_z_min       =
{_v776_fmt(s['sigma_z_min'])} Pa",
        "",
        "EHL",
        f"ue                = {_v776_fmt(c['ue'])} m/s
[ESTIMADO]",
        f"eta0              = {_v776_fmt(c['eta0'])}
Pa.s [ESTIMADO]",
```

```
        f"alpha_p            = {_v776_fmt(c['alpha_p'])}
Pa^-1 [ESTIMADO]",
        f"U                  = {_v776_fmt(e['U'])}",
        f"G                  = {_v776_fmt(e['G'])}",
        f"W                  = {_v776_fmt(e['W'])}",
        f"k                  = {_v776_fmt(e['k'])}",
        f"h_min              = {_v776_fmt(e['hmin'])} m",
        f"h_c                = {_v776_fmt(e['hc'])} m",
        f"Rq                 = {_v776_fmt(e['Rq'])} m",
        f"Lambda             = {_v776_fmt(e['Lambda'])}",
        "",
        "ATRITO / CISALHAMENTO",
        f"mu                 = {_v776_fmt(c['mu'])}
[ESTIMADO]",
        f"F_t                =
{_v776_fmt(f['friction_force'])} N",
        f"P_fric_Coulomb     =
{_v776_fmt(f['friction_power_coulomb'])} W",
        f"tau_lub            = {_v776_fmt(f['tau_lub'])}
Pa",
        f"q_fric             = {_v776_fmt(f['q_fric'])}
W/m2",
        f"P_fric_EHL         =
{_v776_fmt(f['P_fric_ehl'])} W",
        "",
        "TERMICA",
        f"T0                 = {_v776_fmt(c['T0'])} K
[ESTIMADO]",
        f"DeltaT_bulk        = {_v776_fmt(t['dT_bulk'])}
K",
        f"DeltaT_flash       = {_v776_fmt(t['dT_flash'])}
K",
        f"T_total            = {_v776_fmt(t['T_total'])}
```

```
K",
        "",
        "DESGASTE — V7.78 / EHL MISTO",
        f"F_normal           =
{_v776_fmt(w['normal_force'])} N",
        f"L_a                =
{_v776_fmt(w['asperity_load_ratio_percent'])} %",
        f"L_a(fracao)        =
{_v776_fmt(w['asperity_load_fraction'])}",
        f"F_asp              =
{_v776_fmt(w['asperity_force'])} N",
        f"sliding_distance   =
{_v776_fmt(w['sliding_distance'])} m",
        f"V_w                = {_v776_fmt(w['volume'])}
m3",
        f"d_w                = {_v776_fmt(w['depth'])}
m",
        "",
        "PITTING — V7.79",
        f"sigma_Hm          =
{_v776_fmt(result['pitting']['sigma_Hm'])} Pa",
        f"sigma_Hlim        = {('NAO INFORMADO' if
result['pitting']['sigma_Hlim'] is None else
_v776_fmt(result['pitting']['sigma_Hlim']) + ' Pa')}",
        f"S_H               = {('NAO CALCULADO' if
result['pitting']['S_H'] is None else
_v776_fmt(result['pitting']['S_H']))}",
        f"S_H_min           =
{_v776_fmt(result['pitting']['S_H_min'])}",
        f"U_pitting         = {('NAO CALCULADO' if
result['pitting']['U_pitting'] is None else
_v776_fmt(result['pitting']['U_pitting']))}",
        f"fator_carga_limite = {('NAO CALCULADO' if
```

```
result['pitting']['S_H'] is None else
_v776_fmt(result['pitting']['load_factor_to_limit']))}",
        f"pitting_admissible= {result['pitting']
['admissible']}",
        "",
        "MACROPITTING — MARGEM DE CARGA",
        f"fator_carga_macro  = {('NAO CALCULADO' if
macro['load_factor_to_limit'] is None else
_v776_fmt(macro['load_factor_to_limit']))}",
        f"forca_limite_macro = {('NAO CALCULADO' if
macro['force_limit'] is None else
_v776_fmt(macro['force_limit']))} N",
        "",
        "MICROPITTING — MARGEM DE FILME",
        f"h_min              =
{_v776_fmt(micro['h_min'])} m",
        f"h_min_limite       =
{_v776_fmt(micro['h_min_limit'])} m",
        f"Rq_composto        = {_v776_fmt(micro['Rq'])}
m",
        f"Lambda             =
{_v776_fmt(micro['Lambda'])}",
        f"Lambda_min         =
{_v776_fmt(micro['Lambda_min'])}",
        f"S_lambda           =
{_v776_fmt(micro['S_lambda'])}",
        f"micro_admissible   = {micro['admissible']}",
        f"fator_carga_micro  = {('NAO CALCULADO' if
micro['load_factor_to_limit'] is None else
_v776_fmt(micro['load_factor_to_limit']))}",
        f"forca_limite_micro = {('NAO CALCULADO' if
micro['force_limit'] is None else
_v776_fmt(micro['force_limit']))} N",
```

```
        "vida_micro         = NAO PREVISTA PELO MODELO",
        "tempo_1o_micro      = NAO PREVISTO PELO MODELO",
        "",
        "INTEGRIDADE",
        f"U_sigma            =
{_v776_fmt(g['U_sigma'])}",
        f"U_hmin             = {_v776_fmt(g['U_hmin'])}",
        f"U_Lambda           =
{_v776_fmt(g['U_lambda'])}",
        f"U_temperature      =
{_v776_fmt(g['U_temperature'])}",
        f"U_wear             = {_v776_fmt(g['U_wear'])}",
        f"U_global           =
{_v776_fmt(g['U_global'])}",
        f"U_global_N_eval    =
{_v776_fmt(g['U_global_eval'])}",
        "",
        "FADIGA / VIDA",
        f"frequencia         = {_v776_fmt(result['life']
['frequency'])} Hz",
        f"N_avaliacao        = {_v776_fmt(result['life']
['N_eval'])} ciclos",
        f"t_avaliacao        = {_v776_fmt(result['life']
['t_eval'])} s",
        f"N_fadiga           = {_v776_fmt(result['life']
['N_fatigue'])} ciclos",
        f"t_fadiga           = {_v776_fmt(result['life']
['t_fatigue'])} s",
        f"D_fadiga(N_eval)   =
{_v776_fmt(result['fatigue']['D_eval'])}",
        "",
        "VIDA POR DESGASTE",
        f"taxa desgaste      = {_v776_fmt(result['life']
```

```
['wear_depth_rate'])} m/s",
        f"N_desgaste         = {_v776_fmt(result['life']
['N_wear'])} ciclos",
        f"t_desgaste         = {_v776_fmt(result['life']
['t_wear'])} s",
        "",
        "VIDA GLOBAL",
        f"N_lim              = {_v776_fmt(result['life']
['N_lim'])} ciclos",
        f"t_lim              = {_v776_fmt(result['life']
['t_lim'])} s",
        f"mecanismo limite   = {result['life']
['limiting_mechanism']}",
        "",
        "CLASSIFICACAO DAS ENTRADAS",
        "PRIMARIAS: F, R1, R2, E1, E2, nu1, nu2",
        "DERIVADAS: E', R', a, b, area, p_media, p_H,
campo de tensoes",
        "ESTIMADAS: ue, v_slide, eta0, alpha_p, Rq1, Rq2,
T0, mu, beta, parametros termicos, desgaste e limites",
        "",
        "OBSERVACAO",
        "A vida por fadiga depende explicitamente de
sigma_ref, N_ref e m fornecidos no caso. A vida por
desgaste depende da lei de Archard, do limite de
desgaste, da velocidade de deslizamento e da frequencia
de ciclos. Esses parametros permanecem entradas de
referencia e nao sao derivados do contato Hertziano.",
        "",
        "=== FIM V7.81 ===",
    ]
    return "\n".join(lines)
```

```
def run_v776_reference_case():
    result = execute_v776_reference_case()
    report = format_v776_reference_report(result)
    return result, report


if __name__ == "__main__":
    # Execução normal no Pydroid 3: interface Tkinter.
    # O caso de referência continua disponível por
run_v776_reference_case().
    root = tk.Tk()
    try:
        root.option_add("*Font", ("TkDefaultFont", 10))
    except Exception:
        pass
    App(root)
    root.mainloop()
```